

Xbox Game Platform Database

Database Design, SQL Analytics, and Automation in MySQL

Betelhem Asefa

Fall 2025

1. Project Overview

Xbox runs a large digital ecosystem: consoles, Game Pass, the Microsoft Store, Xbox Live, and online multiplayer. Each of these produces data about players, games, purchases, and subscriptions, and it all has to stay consistent and be easy to analyze.

For this project, I designed and built a MySQL database that represents a simplified version of Xbox's backend. I then used it the way an analytics team would: writing SQL to answer business questions, building reusable reporting objects, automating data-integrity rules, and visualizing the results in Python.

The project had three phases:

1. **Design:** business rules, relational schema, normalization, and an EERD
2. **Analytics:** 12 analytical queries, KPI views, and summary tables
3. **Automation and visualization:** stored procedures, functions, triggers, transactions, access roles, and Python charts

Tools: MySQL, MySQL Workbench, phpMyAdmin, Python (pandas, matplotlib, seaborn) in Google Colab

Business questions

- Which games and DLC generate the most revenue and purchases?
- Is spending spread evenly across players, or concentrated among a few?
- How often do players who buy a game also buy its DLC?
- How does subscription adoption differ by region?
- How can procedures, triggers, and views automate reporting and protect data integrity?

2. Database Design

Business rules

1. Every game and DLC is registered once in `Content_Item` and then specialized as either a Game or a DLC.
2. Each game belongs to one developer, and each DLC belongs to one parent game.

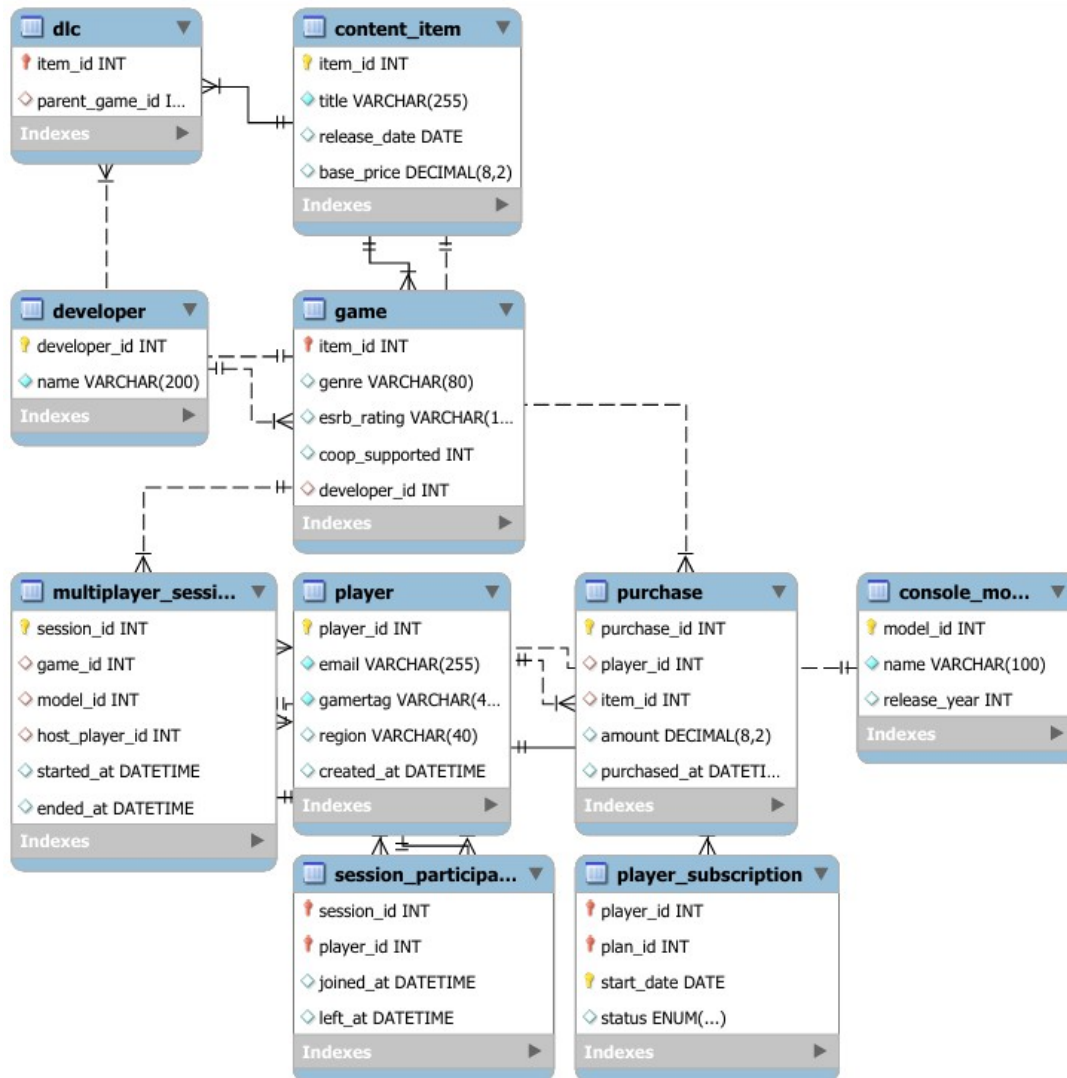
3. Each player must have a unique email and gamertag.
4. Players can hold more than one subscription, but never a duplicate one. Status must be active, paused, or expired.
5. Every purchase must reference a valid player and a valid content item.
6. Every multiplayer session belongs to one game and one console model and has a host. Players can join many sessions, and sessions can have many players.

Schema

Table	Primary key	Foreign keys
Developer	developer_id	none
Content_Item	item_id	none
Game	item_id	item_id → Content_Item; developer_id → Developer
DLC	item_id	item_id → Content_Item; parent_game_id → Game
Player	player_id	none
Console_Model	model_id	none
Subscription_Plan	plan_id	none
Player_Subscription	player_id, plan_id, start_date	player_id → Player; plan_id → Subscription_Plan
Purchase	purchase_id	player_id → Player; item_id → Content_Item
Multiplayer_Session	session_id	game_id → Game; model_id → Console_Model; host_player_id → Player
Session_Participant	session_id, player_id	session_id → Multiplayer_Session; player_id → Player

Normalization: All relations are in third normal form (3NF). Every non-key attribute depends only on its table's full primary key. Game and DLC use the supertype's key as their own primary key (PK = FK), which keeps dependencies key-based. Session_Participant is an associative table with a composite key. Player_Subscription includes start_date in its key so a player's subscription history can be stored without update anomalies.

Integrity testing: I wrote tests that intentionally insert bad data, such as duplicate emails, invalid foreign keys, negative amounts, and sessions that end before they start, to confirm the constraints reject it.



Enhanced Entity-Relationship Diagram (EERD)

3. SQL Analytics

I wrote 12 queries using joins, CTEs, window functions (`RANK()`), CASE logic, and conditional aggregation. The database holds a small sample dataset, so the results below show what each query is built to measure rather than real-world findings.

#	Question	Main techniques	Result in sample data
1	Top games by number of purchases	Multi-table joins	Elden Ring had the most purchases
2	Average session length by console model	CTE, <code>TIMESTAMPDIFF</code> , <code>RANK</code>	All consoles tied at 120 minutes
3	Lifetime spend,	<code>LEFT JOIN</code> , <code>CASE</code>	Top 10 players split

#	Question	Main techniques	Result in sample data
	average order value, and spend tier per player		between Platinum and Gold
4	DLC attach rate per game	Two CTEs, COUNT DISTINCT, NULLIF	Forza Horizon 5 had 100% attach; Elden Ring 33%
5	Developer revenue, including DLC	CTE with UNION ALL, RANK	FromSoftware ranked #1
6	Subscription adoption and active rate by region	CTE, conditional SUM, subquery	DE, JP, and BR were 100% active; US was 40%
7	Days between first and latest purchase	MIN/MAX, TIMESTAMPDIFF	Every player had only one purchase
8	Revenue by ESRB rating	Join, aggregation	M-rated titles brought in the most revenue
9	Developer revenue from base games	RANK	FromSoftware ranked #1
10	Most active multiplayer hosts	Join, aggregation	Every player hosted one session
11	Revenue and orders by game	Multi-table joins	Elden Ring had the highest revenue
12	Purchases and revenue by region	Join, aggregation	The US led in both

The next three queries are the most relevant to commerce analytics, so I've included them in more detail.

Query 3: Player spend, average order value, and spend tiers

This query calculates each player's order count, lifetime spend, and average order value, then assigns a spend tier (Bronze, Silver, Gold, or Platinum) with a CASE statement. Separating high spenders from casual buyers helps the platform focus retention efforts on its most valuable players and use incentives to grow spending among everyone else.

```

-- Q3: Player lifetime spend, purchase count, Average Order Value (AOV), and spend tier
SELECT
  p.player_id,
  p.gamertag,
  COUNT(pr.purchase_id)           AS orders,
  ROUND(SUM(pr.amount), 2)       AS lifetime_spend,
  ROUND(AVG(pr.amount), 2)       AS avg_order_value,
  CASE
    WHEN SUM(pr.amount) >= 69.99 THEN 'Platinum'
    WHEN SUM(pr.amount) >= 49.99 THEN 'Gold'
    WHEN SUM(pr.amount) >= 29.99 THEN 'Silver'
    WHEN SUM(pr.amount) > 0 THEN 'Bronze'
    ELSE 'No Spend'
  END AS spend_tier
FROM Player p
LEFT JOIN Purchase pr ON pr.player_id = p.player_id
GROUP BY p.player_id, p.gamertag
ORDER BY lifetime_spend DESC, orders DESC, p.player_id
LIMIT 10;

-- Insight: There are players at the Platinum tier with disproportionate revenue.
-- This would suggest that targeted retention could be effective.

```

Query 3 code

player_id	gamertag	orders	lifetime_spend	avg_order_value	spend_tier
3	PixelHero	1	69.99	69.99	Platinum
10	SilentHunter	1	69.99	69.99	Platinum
16	AquaWave	1	69.99	69.99	Platinum
22	ZeroKnight	1	69.99	69.99	Platinum
1	GamerOne	1	59.99	59.99	Gold
4	SniperWolf	1	59.99	59.99	Gold
5	SpeedRacer	1	59.99	59.99	Gold
6	ShadowFox	1	59.99	59.99	Gold
7	ArcadeKing	1	59.99	59.99	Gold
8	NovaBlast	1	59.99	59.99	Gold

Query 3 results

Query 4: DLC attach rate

Attach rate is the percentage of a game's buyers who also bought its DLC. I built two CTEs, one for base-game buyers and one for DLC buyers, and divided the distinct counts. NULLIF prevents division by zero. Forza Horizon 5 had a 100% attach rate. Elden Ring had a larger buyer base but only 33% attach, which points to an opportunity to promote DLC bundles to its existing players.

```

-- Q4: DLC attach rate per base game (buyers of base game who also bought its DLC)
WITH base_buyers AS (
    SELECT g.item_id AS game_id, p.player_id
    FROM Game g
    JOIN Purchase p ON p.item_id = g.item_id
),
dlc_buyers AS (
    SELECT d.parent_game_id AS game_id, p.player_id
    FROM DLC d
    JOIN Purchase p ON p.item_id = d.item_id
)
SELECT
    ci.title                                AS game_title,
    COUNT(DISTINCT bb.player_id)           AS game_buyers,
    COUNT(DISTINCT db.player_id)           AS dlc_buyers_for_that_game,
    ROUND(
        100.0 * COUNT(DISTINCT db.player_id)
        / NULLIF(COUNT(DISTINCT bb.player_id),0), 1
    )                                       AS dlc_attach_rate_pct
FROM base_buyers bb
JOIN Content_Item ci ON ci.item_id = bb.game_id
LEFT JOIN dlc_buyers db

```

Query 4 code

game_title	game_buyers	dlc_buyers_for_that_game	dlc_attach_rate_pct
Forza Horizon 5	1	1	100.0
Cyberpunk 2077	2	1	50.0
Destiny 2	2	1	50.0
Halo Infinite	2	1	50.0
Elden Ring	3	1	33.3
Assassins Creed Mirage	2	0	0.0
DOOM Eternal	2	0	0.0
Red Dead Redemption 2	2	0	0.0
Spider-Man 2	2	0	0.0
Starfield	2	0	0.0

Query 4 results

Query 6: Subscription adoption by region

This query measures two things for each region: the share of all players who hold a subscription (adoption), and the share of subscription records that are currently active (active rate). Germany, Japan, and Brazil kept every subscription active, while the US, which has the largest subscriber base, had a 40% active rate. A gap like that suggests regional retention campaigns would be worth testing.

```

-- Q6: Subscription adoption and active rate by region
WITH adoption AS (
  SELECT
    pl.region,
    COUNT(DISTINCT ps.player_id) AS subs_players,
    SUM(ps.status = 'active') AS active_rows, -- counts rows
    COUNT(*) AS total_rows
  FROM Player_Subscription ps
  JOIN Player pl ON pl.player_id = ps.player_id
  GROUP BY pl.region
)
SELECT
  region,
  subs_players,
  total_rows AS sub_records,
  active_rows,
  ROUND(100.0 * active_rows / NULLIF(total_rows,0), 1) AS active_rate_pct,
  ROUND(
    100.0 * subs_players / NULLIF((SELECT COUNT(*) FROM Player),0), 1
  ) AS adoption_pct_of_players
FROM adoption
ORDER BY active_rate_pct DESC, subs_players DESC, region;

```

Query 6 code

region	subs_players	sub_records	active_rows	active_rate_pct	adoption_pct_of_players
DE	3	3	3	100.0	12.0
JP	2	2	2	100.0	8.0
BR	1	1	1	100.0	4.0
CA	4	4	3	75.0	16.0
FR	3	3	2	66.7	12.0
UK	2	2	1	50.0	8.0
US	10	10	4	40.0	40.0

Query 6 results

4. Reporting Objects

To avoid rewriting long queries, I built reusable reporting objects:

- **Views:** `v_game_kpis` shows purchases and revenue per game, with DLC revenue credited back to the parent game. `v_player_kpis` shows each player's game and DLC spending.
- **Summary tables:** `Summary_Game_Daily` stores daily revenue per game, and `Summary_Player_Total` stores lifetime totals per player. Both are refreshed with `TRUNCATE and INSERT...SELECT` so dashboards can read precomputed data quickly.

5. Automation, Integrity, and Security

Object	Name	What it does
Stored procedure	<code>sp_insert_purchase</code>	Checks that the player and

Object	Name	What it does
		item exist and the amount is valid, then inserts the purchase inside a transaction and returns a status message
Stored procedure	sp_get_player_spend_summary	Returns a player's total spend, order count, and first and last purchase dates
Stored procedure	sp_developer_game_revenue	Lists each of a developer's games with its revenue
Function	fn_game_total_revenue	Total revenue for a game, including its DLC
Function	fn_developer_total_revenue	Total revenue for a developer across its games and DLC
Trigger	trigger_purchase_after_insert_audit	Logs every new purchase to an audit table
Trigger	trigger_purchase_integrity	Blocks purchases with a non-positive amount or an item that isn't a game or DLC
Trigger	trigger_multiplayer_session_auto_host	Automatically adds the host as a participant when a session is created

Transactions: I demonstrated START TRANSACTION, SAVEPOINT, ROLLBACK TO, and COMMIT by inserting a valid purchase, setting a savepoint, inserting a mistaken purchase, rolling back only the mistake, and committing the valid one.

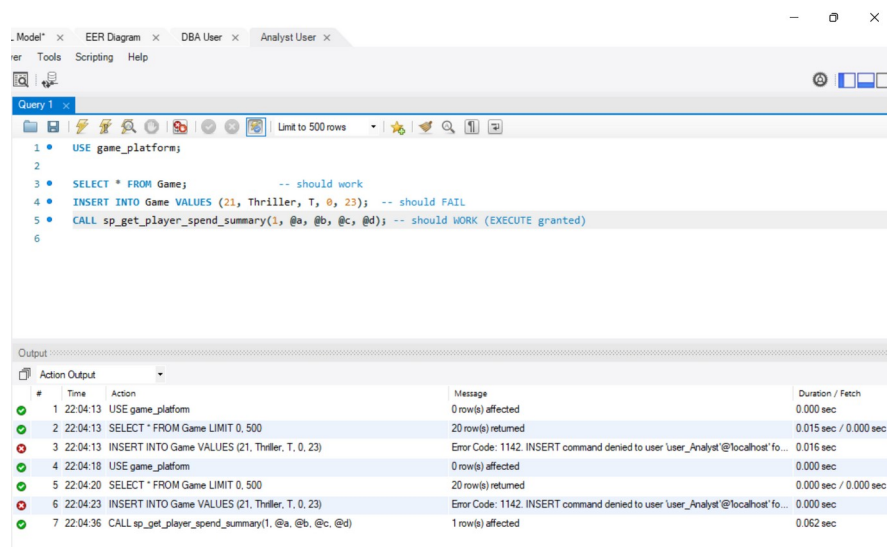
Role-based access: I created two roles. role_DBA has full privileges. role_Analyst can only run SELECT and EXECUTE. When I tested them, the analyst account could query tables and call procedures but was blocked from inserting data.


```

1 • CREATE ROLE 'role_DBA';
2 • CREATE ROLE 'role_Analyst';
3
4 -- DBA role: has full control over game_platform database
5 • GRANT ALL PRIVILEGES ON game_platform.* TO 'role_DBA';
6
7 -- Analyst: can only read and run procedures/functions
8 • GRANT SELECT ON game_platform.* TO 'role_Analyst';
9 • GRANT EXECUTE ON game_platform.* TO 'role_Analyst';
10
11
12 -- Create test users to test privileges
13 • CREATE USER 'user_DBA'@'localhost' IDENTIFIED BY 'PasswordDBA!';
14 • CREATE USER 'user_Analyst'@'localhost' IDENTIFIED BY 'PasswordAnalyst!';
15
16 -- 4) Grant roles to those users
17 • GRANT 'role_DBA' TO 'user_DBA'@'localhost';
18 • GRANT 'role_Analyst' TO 'user_Analyst'@'localhost';
19
20 -- 5) Make the roles active by default
21 • SET DEFAULT ROLE 'role_DBA' TO 'user_DBA'@'localhost';

```

Role creation script



The screenshot shows a database IDE with a query window and an output window. The query window contains the following SQL script:

```

1 • USE game_platform;
2
3 • SELECT * FROM Game; -- should work
4 • INSERT INTO Game VALUES (21, Thriller, T, 0, 23); -- should FAIL
5 • CALL sp_get_player_spend_summary(1, @a, @b, @c, @d); -- should WORK (EXECUTE granted)
6

```

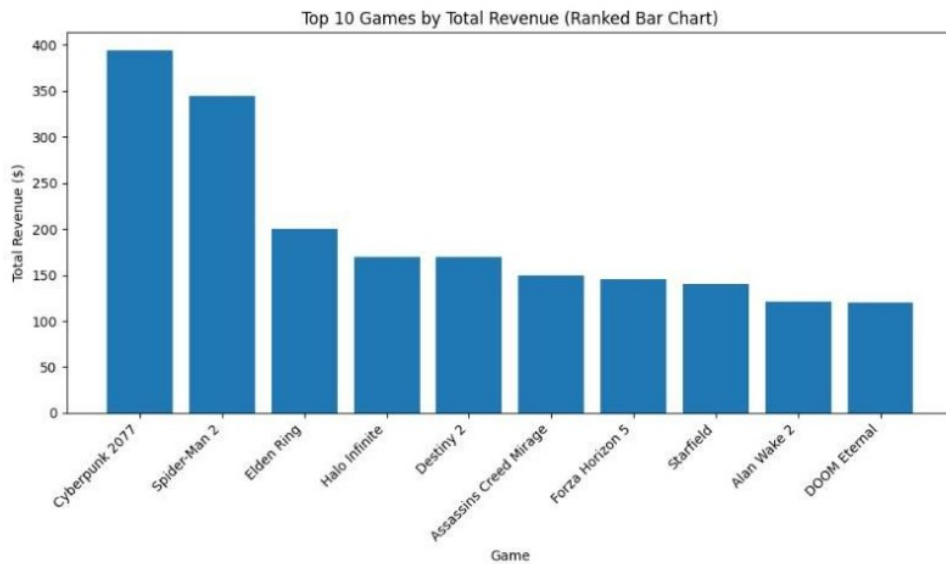
The output window shows the results of the script execution. The output is as follows:

#	Time	Action	Message	Duration / Fetch
1	22:04:13	USE game_platform	0 row(s) affected	0.000 sec
2	22:04:13	SELECT * FROM Game LIMIT 0, 500	20 row(s) returned	0.015 sec / 0.000 sec
3	22:04:13	INSERT INTO Game VALUES (21, Thriller, T, 0, 23)	Error Code: 1142. INSERT command denied to user 'user_Analyst'@'localhost' fo...	0.016 sec
4	22:04:18	USE game_platform	0 row(s) affected	0.000 sec
5	22:04:20	SELECT * FROM Game LIMIT 0, 500	20 row(s) returned	0.000 sec / 0.000 sec
6	22:04:23	INSERT INTO Game VALUES (21, Thriller, T, 0, 23)	Error Code: 1142. INSERT command denied to user 'user_Analyst'@'localhost' fo...	0.000 sec
7	22:04:36	CALL sp_get_player_spend_summary(1, @a, @b, @c, @d)	1 row(s) affected	0.062 sec

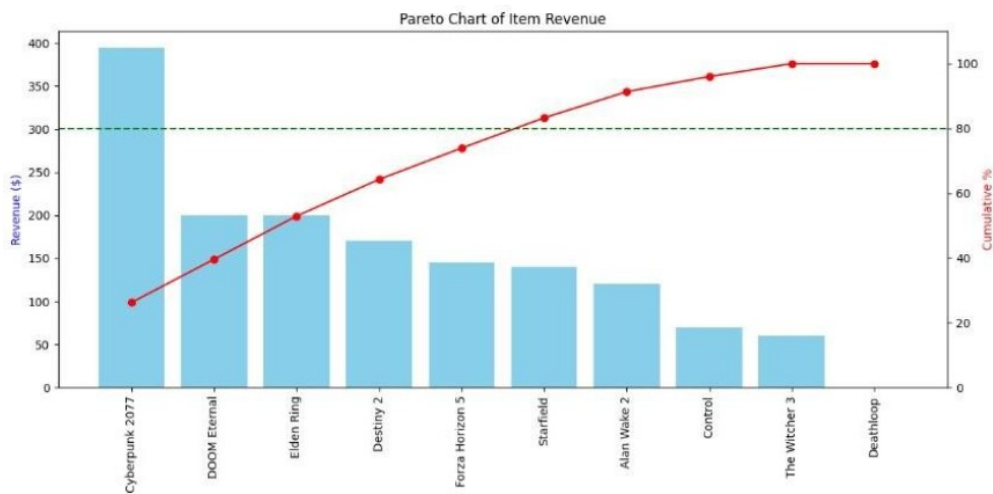
Analyst user: SELECT and procedure calls succeed, INSERT is denied

6. Visualization

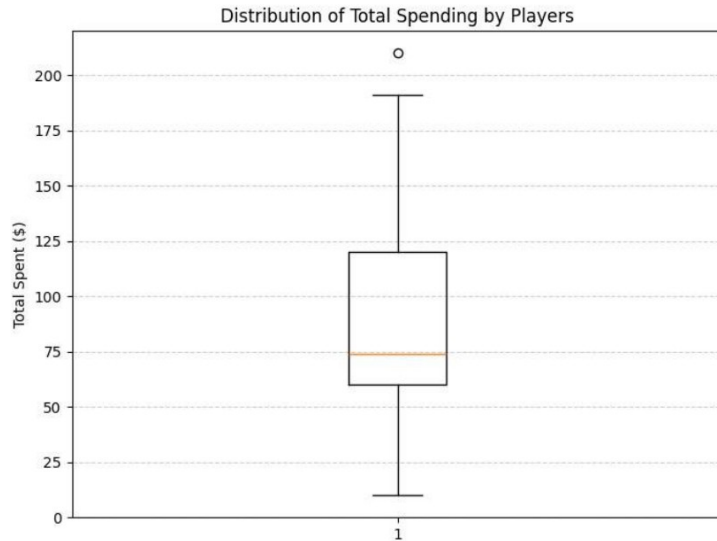
I exported the views and procedure results to CSV and visualized them in Python (pandas, matplotlib, seaborn). These charts reflect the data after the Phase 3 test purchases were added, so the rankings differ slightly from the Phase 2 query results.



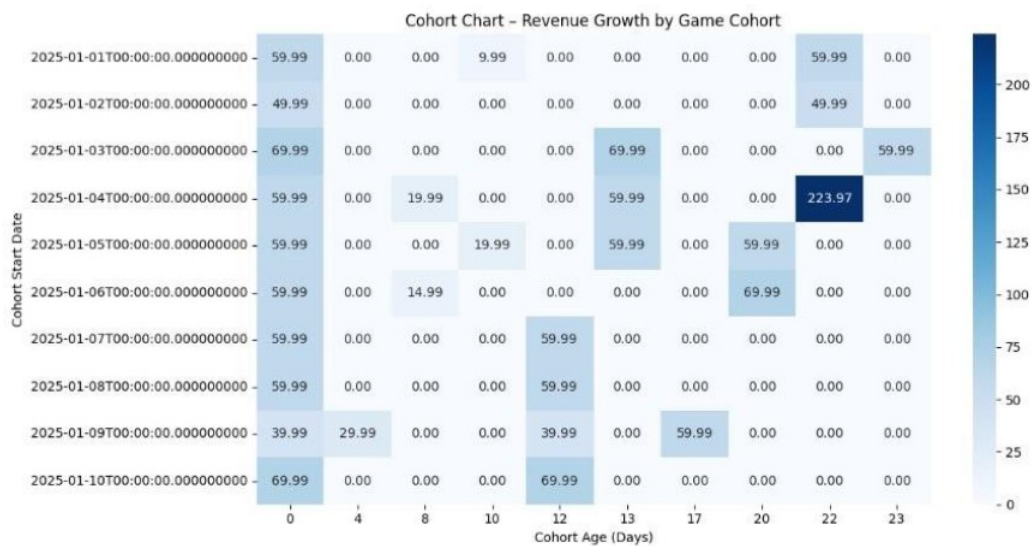
Top 10 games by total revenue. Cyberpunk 2077 and Spider-Man 2 led.



Pareto chart of item revenue. A few titles account for most of the revenue.



Distribution of total spending per player. Most players spent between about \$60 and \$120, with one high-spending outlier.



Revenue cohort heatmap. Revenue for each game cohort by days since its first sale.

7. Insights and Recommendations

Insights

- A small group of games brings in most of the revenue, as the Pareto chart shows.
- DLC attach rates vary widely between games, so add-on content is an underused revenue lever for some titles.
- Subscription activity differs by region, which points to opportunities for region-specific retention.

- No player in the sample made a repeat purchase. On a real platform, that would be a warning sign for recurring revenue.
- The triggers and procedures make sure every purchase is valid and every change is recorded.

Recommendations

- Keep investing in top-performing titles, and use pricing or bundles to help weaker games reach more players.
- Promote DLC bundles to games with large buyer bases and low attach rates.
- Reward high-spending players with exclusive content or early access, and give one-time buyers small incentives to make a second purchase.
- Tailor subscription marketing by region, starting with regions that have low active rates.
- Keep the auditing triggers and validation procedures in place, since every decision depends on accurate data.